

Doc. no. D1793R0
Date: 2019-06-17
Project: Programming Language C++
Audience: Evolution Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Simplifying Contract Syntax

Table of Contents

1. [Revision History](#)
 - [Revision 0](#)
2. [1 Introduction](#)
3. [2 Stating the problem](#)
4. [3 Other Directions](#)
5. [4 Formal Wording](#)
6. [5 References](#)

Revision History

Revision 0

Original version of the paper for the 2019 pre-Cologne mailing.

1 Introduction

The grammar for contracts is a little more verbose than it needs to be. This paper recommends we take this last change to simplify, before the standard is finalized. This paper proposes no functional changes, merely simplification of how we express the current semantics.

2 Stating the problem

The grammar for contracts reflects the evolutions of the design, and a few reasonable use cases appear more verbose than they need to be. As we are going to live with this syntax choice for the next few decades, we should take the opportunity to simplify and avoid future embarrassment.

The use case is simple. When we declare pre and post conditions, we assume the default checking level, leaving simple syntax for the common case:

```
int test(int value)
  [[pre: value > 0]]
  [[post result: result < 0]]
{
  return -value;
}
```

Similarly, for assert contract checks, we can assume the default checking level:

```
int test(int value)
  [[pre: value > 0]]
```

```

    [[post result: result < 0]]
  {
    [[assert: value < 100]];

    return -value;
  }

```

However, when it comes to asserting audit and axioms checks, we end up with two tokens:

```

int test(int value)
  [[pre: value > 0]]
  [[post result: result < 0]]
{
  [[assert audit: value < 100]];
  [[assert axiom: value > 0]];

  return -value;
}

```

Notice that `assert` is not disambiguating anything here, and could as easily be inferred as the default checking level. Hence, this paper proposes simplifying the grammar in this case to omit the `assert`

```

int test(int value)
  [[pre: value > 0]]
  [[post result: result < 0]]
{
  [[assert: value > 0]];
  [[audit: value < 100]];
  [[axiom: value != 0]];

  return -value;
}

```

Note that `assert` appears more like `audit` and in this context, which leads to the second suggestion: rename the default checking level to `assert`. This would clean up a remarkable cause of confusion I have hear in many discussions of the contract facility, where different uses of the term 'default' lead to people arguing at cross-purposes to each other.

With this final change, the grammar cleanup is relatively straight forward, see proposed wording below.

3 Other Directions

The main purpose of `pre` and `post` is to disambiguate their different semantic intent when adding a contract checking attribute to a function declaration. If we were to make the optional *result* field mandatory, that would also serve to disambiguate, and allow us to further simplify by proving only a contract checking level in all cases.

```

int test(int value)
  [[assert: value > 0]]
  [[axiom result: result < 0]]
{
  [[assert audit: value < 100]];
  [[assert axiom: value > 0]];

  return -value;
}

```

While this might have been considered 12-18 months ago, that seems too radical a departure from our established convention at this point. Further, `pre` and `post` provide helpful syntactic cues to the reader. They may

be strictly superfluous in the analysis above, but they still provide value, and retain more of the design space for future extensions - otherwise we are restricted on giving meaning to special identifiers in the *result* position, that would become ambiguous with a pre-condition using that special identifier.

4 Formal Wording

Make the following changes to the specified working paper:

9.11.4.1 Syntax [dcl.attr.contract.syn]

```
contract-attribute-specifier :  
[ [ expects contract-levelopt : conditional-expression ] ]  
[ [ ensures contract-levelopt identifieropt : conditional-expression ] ]  
[ [ assert contract-levelopt : conditional-expression ] ]  
  
contract-level :  
assert default  
audit  
axiom
```

5 References

- [N4810](#) Working Draft, Standard for Programming Language C++, Richard Smith